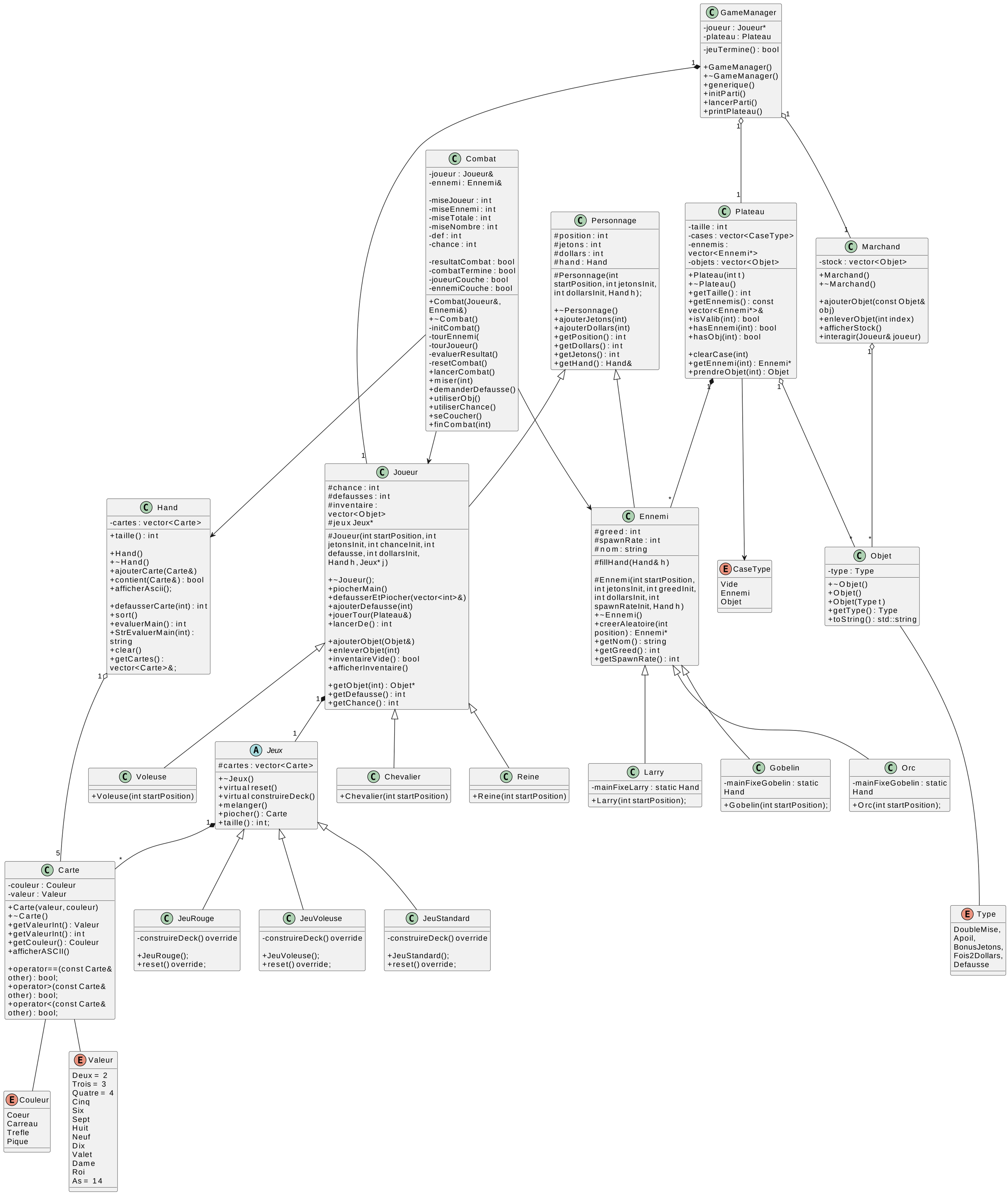
GameManager
-joueur : Joueur*
-plateau : Plateau
-jeuTermine() : bool
+GameManager()
+~GameManager()
+generique()
+initParti()
+lancerParti()
+printPlateau()
Combat
-joueur : Joueur&
-ennemi : Ennemi&
-miseJoueur : int
-miseEnnemi : int
-miseTotale : int
-miseNombre : int
-def : int
-chance : int
-resultatCombat : bool
-combatTermine : bool
-joueurCouche : bool
-ennemiCouche : bool
+Combat(Joueur&, Ennemi&)
+~Combat()
-initCombat()
-tourEnnemi(
-tourJoueur()
-evaluerResultat()
-resetCombat()
+lancerCombat()
+miser(int)
+demanderDefausse()
+utiliserObj()
+utiliserChance()
+seCoucher()
+finCombat(int)
Personnage
#position : int
#jetons : int
#dollars : int
#hand : Hand
#Personnage(int startPosition, int jetonsInit, int dollarsInit, Hand h);
+~Personnage()
+ajouterJetons(int)
+ajouterDollars(int)
+getPosition() : int
+getDollars() : int
+getJetons() : int
+getHand() : Hand&
Plateau
-taille : int
-cases : vector<CaseType>
-ennemis : vector<Ennemi*>
-objets : vector<Objet>
+Plateau(int t)
+~Plateau()
+getTaille() : int
+getEnnemis() : const vector<Ennemi*>&
+isValib(int) : bool
+hasEnnemi(int) : bool
+hasObj(int) : bool
+clearCase(int)
+getEnnemi(int) : Ennemi*
+prendreObjet(int) : Objet
Marchand
-stock : vector<Objet>
+Marchand()
+~Marchand()
+ajouterObjet(const Objet& obj)
+enleverObjet(int index)
+afficherStock()
+interagir(Joueur& joueur)
Hand
-cartes : vector<Carte>
+taille() : int
+Hand()
+~Hand()
+ajouterCarte(Carte&)
+contient(Carte&) : bool
+afficherAscii();
+defausserCarte(int) : int
+sort()
+evaluerMain() : int
+StrEvaluerMain(int) : string
+clear()
+getCartes() : vector<Carte>&;
Joueur
#chance : int
#defausses : int
#inventaire : vector<Objet>
#jeux Jeux*
#Joueur(int startPosition, int jetonsInit, int chanceInit, int defausse, int dollarsInit, Hand h, Jeux* j)
+~Joueur();
+piocherMain()
+defausserEtPiocher(vector<int>&)
+ajouterDefausse(int)
+jouerTour(Plateau&)
+lancerDe() : int
+ajouterObjet(Objet&)
+enleverObjet(int)
+inventaireVide() : bool
+afficherInventaire()
+getObjet(int) : Objet*
+getDefausse() : int
+getChance() : int
Ennemi
#greed : int
#spawnRate : int
#nom : string
#fillHand(Hand& h)
#Ennemi(int startPosition, int jetonsInit, int greedInit, int dollarsInit, int spawnRateInit, Hand h)
+~Ennemi()
+creerAleatoire(int position) : Ennemi*
+getNom() : string
+getGreed() : int
+getSpawnRate() : int
CaseType
Vide
Ennemi
Objet
Objet
-type : Type
+~Objet()
+Objet()
+Objet(Type t)
+getType() : Type
+toString() : std::string
Jeux
#cartes : vector<Carte>
+~Jeux()
+virtual reset()
+virtual construireDeck()
+melanger()
+piocher() : Carte
+taille() : int;
Voleuse
+Voleuse(int startPosition)
Chevalier
+Chevalier(int startPosition)
Reine
+Reine(int startPosition)
Larry
-mainFixeLarry : static Hand
+Larry(int startPosition);
Gobelin
-mainFixeGobelin : static Hand
+Gobelin(int startPosition);
Orc
-mainFixeGobelin : static Hand
+Orc(int startPosition);
Carte
-couleur : Couleur
-valeur : Valeur
+Carte(valeur, couleur)
+~Carte()
+getValeurInt() : Valeur
+getValeurInt() : int
+getCouleur() : Couleur
+afficherASCII()
+operator==(const Carte& other) : bool;
+operator>(const Carte& other) : bool;
+operator<(const Carte& other) : bool;
JeuRouge
-construireDeck() override
+JeuRouge();
+reset() override;
JeuVoleuse
-construireDeck() override
+JeuVoleuse();
+reset() override;
JeuStandard
-construireDeck() override
+JeuStandard();
+reset() override;
Couleur
Coeur
Carreau
Trefle
Pique
Valeur
Deux = 2
Trois = 3
Quatre = 4
Cinq
Six
Sept
Huit
Neuf
Dix
Valet
Dame
Roi
As = 14
Type
DoubleMise,
Apoil,
BonusJetons,
Fois2Dollars,
Defausse